

SURVEY

SMART: Semantic, Multi-Objective, and Reinforcement-Based Adversarial Training for Email Spam Detection

KAMAL TAHA^{1b}, (Senior Member, IEEE)

Department of Computer Science, Khalifa University, Abu Dhabi, United Arab Emirates

e-mail: kamal.taha@ku.ac.ae

ABSTRACT This paper introduces SMART, a robust framework for email spam detection designed to adapt to and mitigate adversarial spam emails. SMART integrates several advanced techniques, including text cleaning and tokenization, word embeddings, K-means clustering for semantic enhancement, multiobjective adversarial training, and reinforcement learning for dynamic spam detection. By leveraging these components, SMART enhances the accuracy and efficiency of detecting and classifying spam emails, even in the face of evolving adversarial tactics. The framework incorporates semantic enrichment using BERT for contextual word replacements and reinforcement learning to adapt spam detection strategies dynamically. SMART provides a comprehensive solution to address the limitations of traditional spam detection methods, ensuring more secure and reliable email communication. We compared SMART experimentally with the following comparable methods: EGOAMLPs, MFO, OPTICS, STM, and MOBGOA. The following is average percentage improvement of SPAM over each of these methods: 2.41% over; 3.36% over; 4.51% over; 6.16% over; 6.89% over.

INDEX TERMS Email spam detection, spam filtering, text classification, adversarial training, reinforcement learning, BERT.

I. INTRODUCTION

Recently, the issue of unwanted commercial bulk emails, known as spam, has become a significant problem on the internet. The individuals responsible for sending these spam messages are referred to as spammers. These spammers collect email addresses from various websites, chatrooms, and viruses [6]. Spam hinders users from making optimal use of their time, storage capacity, and network bandwidth.

The large volume of spam emails circulating through computer networks has detrimental effects on the memory space of email servers, communication bandwidth, CPU power, and user time [7]. The incidence of spam email increases annually and now accounts for over 77% of global email traffic [8]. Recipients of unsolicited spam emails often find them extremely irritating.

The associate editor coordinating the review of this manuscript and approving it for publication was Yassine Maleh^{1b}.

Also, many users suffer financial losses due to internet scams and other fraudulent activities perpetrated by spammers who disguise their emails as being from reputable companies to extract sensitive personal information like passwords, Bank Verification Numbers, and credit card numbers.

Spam also infiltrates written reviews of products on social networking sites. According to Liu and Pang [9], approximately 30–35% of online reviews are considered spam. These spam reviews aim to manipulate people's purchasing decisions and affect product ratings [10], [11]. Consequently, detecting fake reviews has become a significant concern, as online review systems risk becoming completely ineffective if this critical issue is not addressed [12].

To effectively combat email spam, leading email providers such as Gmail, Yahoo Mail, and Outlook employ various machine learning (ML) techniques, including Neural Networks, in their spam filters. These ML techniques can learn to identify spam and phishing messages by analyzing large

volumes of such messages across extensive computer networks. Because machine learning can adapt to changing conditions, Gmail and Yahoo Mail spam filters do more than just check junk emails against pre-existing rules; they generate new rules based on their ongoing learning from spam filtering operations. The rise in email communication has led to a significant increase in spam emails, posing threats such as phishing, malware distribution, and fraud.

In the field of spam detection and classification, numerous methods have been established [13], [14]. Traditional approaches include techniques such as character matching, the naive Bayes algorithm [15], support vector machines (SVM) [16], and ensemble learning [17]. With the advent of deep learning, various deep models like Convolutional Neural Networks (CNN), Recurrent Neural Networks (RNN), and BERT have been applied to spam recognition [18], [19], [20].

For example, [21] proposed a comprehensive dual-layer architecture for phishing and spam email detection, while [22] developed a hierarchical attention hybrid deep learning method for spam email detection.

Below are different categories of spam filtering techniques widely applied to address the problem of email spam:

- *Content-Based Filtering Technique:* This technique uses machine learning approaches such as Naïve Bayesian classification, Support Vector Machines, K-Nearest Neighbors, and Neural Networks to create automatic filtering rules and classify emails. It analyzes the words, occurrences, and distributions of words and phrases in email content to filter incoming spam [23].
- *Case-Based Spam Filtering Method:* This popular method involves extracting both spam and non-spam emails from each user's email collection. The emails undergo pre-processing steps like feature extraction, selection, and grouping of email data. The data is classified into two vector sets, and machine learning algorithms train and test the datasets to determine whether incoming emails are spam or non-spam [23].
- *Heuristic or Rule-Based Spam Filtering Technique:* This approach uses pre-established rules or heuristics to assess numerous patterns, typically regular expressions, against a message. The score of a message increases with more matching patterns and decreases if patterns do not match. Messages exceeding a specific score threshold are filtered as spam. Some rules remain constant, while others require regular updates to address new spam messages effectively. An example of a rule-based spam filter is SpamAssassin [24].
- *Previous Likeness-Based Spam Filtering Technique:* This method uses memory-based or instance-based machine learning methods to classify incoming emails based on their similarity to stored examples (e.g., training emails). The attributes of the email create a multi-dimensional space vector to plot new instances, which are then classified based on the closest training instances. The k-nearest neighbor (kNN) algorithm is used for filtering spam emails [25].

- *Adaptive Spam Filtering Technique:* This method detects and filters spam by grouping emails into different classes, each with a representative text. Each incoming email is compared to these groups, and a similarity percentage determines the probable group to which the email belongs [26].

The necessity for improved spam detection methods has been highlighted by several studies, underscoring the limitations of static rule-based systems in countering advanced spam tactics [27], [28]. The applications of BERT for nuanced language analysis in spam detection show promising results in recent research [29], [30].

We introduce in this paper a robust framework for email spam detection that can adapt to and mitigate the impact of adversarial spam emails. It is named **SMART** “Semantic, Multi-objective, And Reinforcement-based Adversarial Training”. SMART integrates several key components: text cleaning and tokenization, word embeddings, K-means clustering for semantic enhancement, multi-objective adversarial training, and reinforcement learning for dynamic spam detection. Each component plays a crucial role in enhancing the system's ability to detect and classify spam emails accurately and efficiently.

A. MOTIVATION OF THE PROPOSED SYSTEM

Traditional spam detection techniques, which rely on static rules and conventional machine learning models, struggle to keep up with the sophisticated and evolving nature of spam emails. SMART aims to address these limitations by developing a robust spam detection framework that integrates multi-objective adversarial training [31], semantic enrichment [32], and reinforcement learning [33] using BERT [34].

SMART aims to enhance the system's ability to accurately and efficiently detect and classify spam emails, even as spammers continuously adapt their tactics. The study focuses on improving the adaptability and resilience of spam detection models to maintain effective protection against spam threats, thereby ensuring the security and reliability of email communication.

B. SYSTEM ARCHITECTURE AND OUTLINE OF THE APPROACH

Fig. 1 shows SMART's system architecture for adversarial email detection that enhances spam identification through a multi-layered approach. As shown in the figure, the architecture begins with text cleaning and tokenization using NLTK, followed by converting words into vectors with pre-trained Word2Vec embeddings. These embeddings are then clustered using K-means to form semantically coherent groups, which serve as the foundation for a robust semantic enhancement layer. In this layer, words are replaced with contextually appropriate synonyms to ensure consistency and readability, leveraging BERT's capabilities for nuanced language analysis.

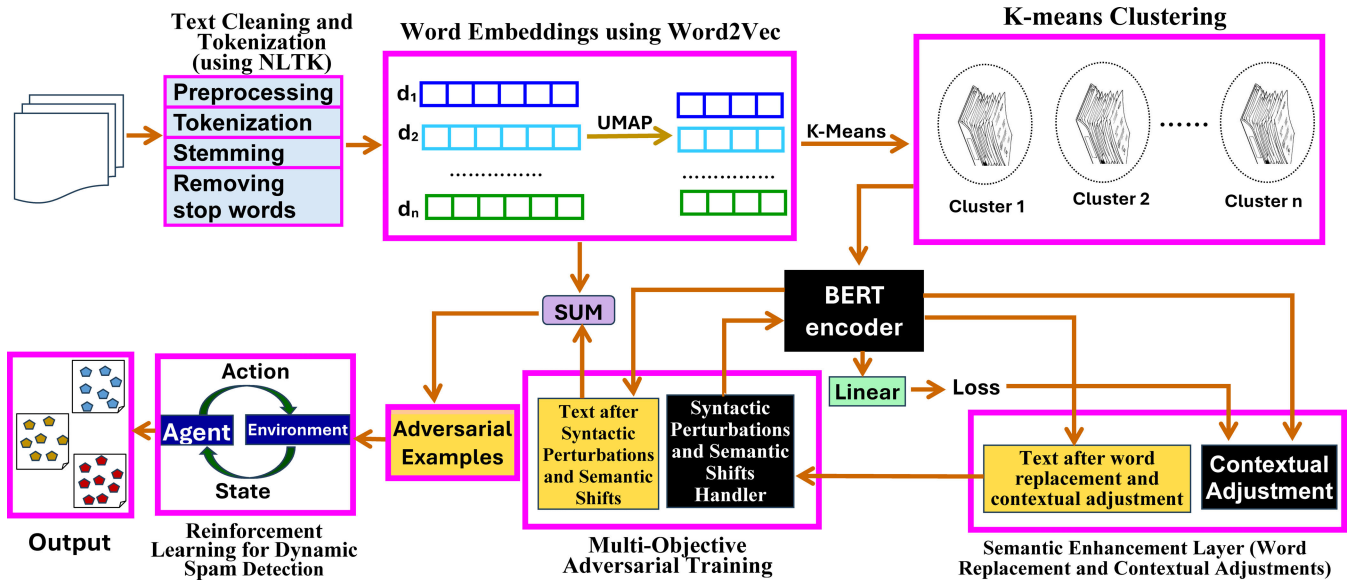


FIGURE 1. The system architecture of our proposed framework SMART.

Multi-objective adversarial training involves introducing syntactic perturbations and semantic shifts to the training data, challenging the model to recognize and classify spam despite these variations. Tools like NLTK and SpaCy are used to maintain model robustness.

A key innovation is the integration of reinforcement learning for dynamic spam detection. The spam detection model acts as an agent, classifying emails and updating its policy based on rewards from classification accuracy. This adaptive approach ensures that the model remains effective in identifying spam, even as adversarial tactics evolve.

By integrating these advanced techniques, SMART aims to provide a comprehensive solution for robust and adaptive spam detection, addressing the limitations of traditional methods and enhancing the security and reliability of email communication.

The following are the specific components of SMART's system architecture, as illustrated in Fig. 1:

- (a) **Word Embeddings (Section II):** Converts words into vectors using pre-trained Word2Vec.
- (b) **K-means Clustering (Section III):** Groups word embeddings into semantically coherent clusters.
- (c) **Semantic Enhancement Layer (Section IV):**
 - *Word Replacement:* Replaces words with synonyms from the same cluster to maintain semantic consistency and readability.
 - *Contextual Adjustments:* Uses BERT to ensure replacements fit contextually.
- (d) **Multi-Objective Adversarial Training (Section V):**
 - *Syntactic Perturbations* (Manages variations in sentence structure using NLTK libraries).
 - *Semantic Shifts* (Handles changes in word choice and context using SpaCy libraries).

(e) Reinforcement Learning (RL) for Dynamic Spam Detection (Section VI):

- *Agent:* The spam detection model acts as the agent interacting with email data.
- *Initialization:* The agent starts with an initial state using the current spam detection policy.
- *Action Selection:* Guided by its policy, the agent classifies emails as spam or legitimate.
- *Environment Response:* The email data provides a new state and a reward signal based on classification accuracy (positive for correct, negative for incorrect).
- *Learning:* The agent updates its policy to maximize long-term rewards, improving its ability to identify spam, including adversarial examples.

II. RELATED WORK

A. DEEP LEARNING BASED METHODS

Ghaleb et al. [1] proposed a new Spam Detection System (SDS) that combines six variants of the enhanced Grasshopper Optimization Algorithm (EGOA) with a Multilayer Perceptron (MLP), forming Neural Network (NN) models called EGOAMLPs. These models are trained using EGOAs to classify emails as spam or non-spam and are evaluated on three benchmark datasets: SpamBase, SpamAssassin, and UK-2011 Webspam. The results demonstrate the effectiveness of the EGOAMLPs in detecting diverse forms of spam email.

Seth and Biswas [18] introduced Deep Learning techniques, specifically CNN, to address spam emails containing both images and text. They proposed two multimodal architectures designed to classify emails by integrating image and text classifiers, which then produce an output class.

TABLE 1. Comparison between SMART and other related techniques.

Ref.	Description of the article	Main technique employed	Differences SMART and this technique	Why SMART is better
[39]	The article explores the vulnerability of spam filters to adversarial NLP attacks such as synonym replacement, ham word injection, and spam word spacing. These techniques deceive machine learning-based filters by subtly modifying text.	Adversarial NLP attacks on Naive Bayesian spam filters (synonym replacement, word injection, spacing).	Focuses on attacking spam filters, while SMART focuses on building a robust defense.	SMART defends against such attacks using semantic analysis (BERT) and reinforcement learning.
[40]	The article studies crafting adversarial spam email content to bypass machine learning-based spam detectors using perturbations in the feature space. It proposes two novel techniques to increase the transferability and success rate of such attacks.	Adversarial content crafting using PGD in feature space, with improved text transformation.	Focuses on crafting adversarial emails, while SMART integrates semantic analysis and RL for defense.	SMART is a defense model combining adversarial training with semantic awareness and dynamic adaptation.
[41]	This article presents AdverSPAM, an AML-based method targeting spam account detection in social networks by optimizing adversarial samples while preserving feature correlations and semantics. It achieves high evasion effectiveness while maintaining realism.	Adversarial sample generation preserving statistical and semantic properties for social network spam detection.	Targets social network accounts, not emails; emphasizes evasion, while SMART aims at dynamic spam detection.	SMART applies to email spam and integrates multiple robust components rather than evasion-specific attacks.
[42]	The article introduces a compound data poisoning technique targeting transformer-based sentiment classifiers. It combines label flipping with adversarial artifact insertion, achieving high poisoning efficacy while remaining stealthy.	Compound data poisoning of transformer-based models via label flipping and stealthy artifact insertion.	Focuses on poisoning transformer sentiment models, not defending spam detection models.	SMART does not rely on poisoning but proactively defends with adaptive training and BERT enhancement
[43]	The study proposes a method to generate adversarial emails using feature perturbation and map them back to realistic 'magic words' in the email text. It evaluates the approach across various ML models and adversarial scenarios.	Gradient-based adversarial perturbation with NLP mapping for spam email generation and evasion.	Focuses on reverse-engineering perturbations into email text; SMART uses BERT and RL for active defense.	SMART handles adversarial threats with more sophisticated semantic and adaptive learning techniques.
[44]	This paper proposes a Transformer-based model for SMS spam detection, modifying the standard architecture for text classification. It shows strong performance compared to CNN, LSTM, and traditional classifiers.	Transformer-based SMS spam classification using attention mechanisms.	Focuses on SMS spam detection with Transformers; SMART deals with adversarial email spam.	SMART handles adversarial email spam with semantic enrichment and adaptive detection strategies
[45]	The article presents AESMOTE, combining reinforcement learning with SMOTE for anomaly detection in imbalanced datasets. It addresses class imbalance and adaptivity in intrusion detection systems, especially for IoT.	Reinforcement Learning combined with SMOTE for class-imbalance-aware anomaly detection.	Targets anomaly detection in IoT, not email; uses RL + SMOTE for class imbalance.	SMART addresses adversarial email threats rather than class imbalance, providing more relevant protection
[46]	The article introduces a dynamic evolving neural network based on reinforcement learning for detecting phishing emails. It adapts online using real-time updates, tackling zero-day phishing attacks with high accuracy	Reinforcement Learning with Dynamic Evolving Neural Networks for phishing email detection	Focuses on detecting phishing emails with evolving NN; SMART combines multi-objective training + RL for general spam.	SMART provides a broader, more robust framework for email spam detection with reinforcement learning and semantic awareness.

Zhang et al. [35] presented an explainable framework for detecting image spam using a CNN model enhanced with XAI methods like SHAP and LIME. The XAI techniques were applied post-hoc to improve the transparency and explainability of the CNN's decisions, boosting user trust in the model. The approach was evaluated on a dataset of 6,636 images.

Mambina et al. [36] explored the use of deep-learning models to filter Swahili SMS spam using a dataset of 32,259 messages from Tanzanian telecommunications companies. The study tested five models—CNN, LSTM, GRU, BiLSTM, and BERT—finding that the CNN-LSTM-LSTM model performed best on Swahili data, while the CNN-BiLSTM excelled on the UCI English dataset. The research highlights the efficacy of deep learning in handling linguistically unique spam detection tasks.

Labonne and Moran [37] demonstrated the superiority of large language models such as Flan-T5, RoBERTa, and SetFit in few-shot spam classification tasks, highlighting

their robustness in low-data scenarios. Their Spam-T5 model, based on Flan-T5, outperformed several baselines across four datasets, showing strong generalization even in adversarial conditions.

Jamal et al. [38] proposed an improved Transformer-based spam detection model (IPSDM) fine-tuned on spam and phishing emails. Their study demonstrates how customized variants of DistilBERT and RoBERTa enhance spam detection accuracy, especially on unbalanced datasets.

Li et al. [39] introduced SpamDam, a framework that focuses on adversary-resistant spam detection using federated learning and adversarial robustness evaluation. Their study reveals the vulnerabilities of models trained on outdated datasets and proposes novel reverse backdoor attacks.

Uddin et al. [40] emphasized the importance of explainability in spam detection systems by integrating XAI techniques with RoBERTa. Their ExplainableDetector enhances transparency in decision-making.

Table 1 presents a comparison between SMART and other related techniques [4], [41], [42], [43], [44], [45], [46], [47]. The following summary highlights the key differences and explains why SMART offers a superior approach:

The SMART framework stands out from other techniques by prioritizing robust **defense** against adversarial email spam through a combination of semantic analysis using BERT, adversarial training, and reinforcement learning. While most other techniques focus on crafting or simulating adversarial attacks—such as synonym manipulation, feature perturbation, or data poisoning—SMART is designed to **resist and adapt** to these threats in real-time. Unlike models that target specific use cases like SMS spam, social network accounts, or IoT anomalies, SMART offers a **comprehensive and generalizable** approach tailored for email spam. Its integration of semantic awareness allows it to better understand and detect subtle manipulations, while reinforcement learning ensures continuous adaptation. This makes SMART not only more effective at handling evolving adversarial strategies but also more scalable and versatile across spam detection scenarios.

B. UNSUPERVISED LEARNING BASED METHODS

Karim et al. [3] proposed an unsupervised clustering approach to differentiate ham from spam emails using a dataset of 22,000 entries based on email body and subject header. The dataset was reduced to ten key features, seven of which were novel and engineered to capture unique email characteristics. Feature selection techniques, including MCFS and Laplacian, were employed to enhance the clustering process by removing low-impact features. Basavaraju and Prabhakar [48] proposed a system that utilizes text clustering based on the Vector Space Model (VSM). The method demonstrated reasonable effectiveness in identifying spam emails.

Laorden et al. [49] proposed a content-based approach for spam filtering, which leverages anomaly detection by comparing features such as ‘Word Frequency’ against a dataset of legitimate (ham) emails. Halder et al. [50] applied clustering algorithms, including K-Means and Expectation Maximization, to analyze schemas based on the stylistic features of emails, such as the total number of punctuations, contractions, and email IDs in the body.

C. SEMI-SUPERVISED LEARNING BASED METHODS

Sedhai and Sun [51] proposed a semi-supervised spam detection framework (S3D) with two modules: a real-time spam detection module and a batch-mode model update module. The real-time module uses four lightweight detectors to label tweets based on blacklisted domains, near-duplicates, trusted users, and a multi-classifier for the remaining tweets. Although the system effectively captures new spamming patterns in real-time, it faces limitations at the tweet level, while more robust features can be derived from user accounts and historical data.

Yerima and Bashar [52] proposed an SMS spam detection system using a semi-supervised novelty detection approach with a One Class SVM (OC-SVM) classifier. The system is trained solely on non-spam data, allowing it to function without labeled spam examples. Evaluated on a dataset of 747 spam and 4,827 non-spam messages, the model uses pre-processed and embedded non-spam data for OC-SVM training. Smadi et al. [41] introduced a framework called the ‘Phishing Email Detection System (PEDS),’ which combines supervised and unsupervised techniques with reinforcement learning. This integration enhances the system’s ability to adapt to changes and modifications.

D. SUPERVISED LEARNING BASED METHODS

Large et al. [53] also suggested that heterogeneous ensemble-based spam filtering frameworks tend to perform better. However, they argued that instead of relying on simpler tree-based ensemble techniques, as used in [54], more advanced methods, such as Random Forest, Rotation Forest, and Deep Neural Networks, can achieve superior performance across various scenarios.

Nizamani et al. [55] stated that effective and advanced feature selection plays a more critical role than the choice of classification algorithms when it comes to identifying deceitful emails. The authors utilized SVM, J48 Decision Tree, and Naive Bayes (NB) classifiers. Their findings highlight that frequency-based features generally yielded the highest accuracy. Islam and Abawajy [56] developed a multitier classifier system where an email is first evaluated for accurate labeling in the initial two tiers. If any misclassification occurs, or if the first two tiers produce conflicting labels, the email is then processed in a third tier. The selection of algorithms (SVM, AdaBoost, and Naive Bayes) for each tier was determined by experimenting with different combinations of algorithms and their respective tiers.

E. SUPPORT VECTOR MACHINE BASED METHODS

Diale et al. [57] demonstrated that when using SVM for email classification, optimizing the kernel type and kernel parameters is crucial. They highlighted that different feature extraction and selection techniques often necessitate the use of varying kernel functions to achieve optimal performance.

Amayri and Bouguila [58] proposed an integrated model that simultaneously utilizes both textual and visual (image) information from emails for spam detection. Their framework is based on constructing probabilistic SVM kernels derived from a mixture of Langevin distributions [58].

F. BIO-INSPIRED INTELLIGENCE BASED METHODS

Ruano-Ordás et al. [59] argued that the application of automatically generated regular expressions (regex) is a particularly powerful method for identifying messages that have been obfuscated by spammers. Their work effectively illustrates how groups of sentences from compromised emails

follow suspicious patterns, making this approach suitable for deployment as a local content-based filtering system.

Zhu and Tan [60] proposed a model inspired by the ‘Biological Immune System (BIS)’ that utilizes a ‘Local Concentration (LC) based Feature Extraction’ approach for developing an anti-spam model. This LC approach is designed to effectively capture position-correlated information within a message by transforming each segment of the message into a corresponding LC feature.

Example 1: Consider an email with the sentence: “Buy cheap meds now.” In the process of generating word embeddings, each word in this sentence (“Buy,” “cheap,” “meds,” “now”) is converted into its corresponding vector representation using a pre-trained embedding model, such as Word2Vec. This transformation involves mapping each word to a high-dimensional vector that encapsulates its semantic properties. Let’s break down the process:

1. **Tokenization:** The sentence is first tokenized into individual words: [“Buy”, “cheap”, “meds”, “now”].
2. **Embedding:** Each token is then transformed into a vector. For instance:
 - “Buy” might be represented as a vector [0.2, -0.3, 0.1, ...]
 - “cheap” might be represented as a vector [0.5, 0.8, -0.6, ...]
 - “meds” might be represented as a vector [0.7, -0.1, 0.4, ...]
 - “now” might be represented as a vector [-0.2, 0.6, -0.4, ...]
3. **Matrix Construction:** These vectors are then combined to form the following matrix:

$$\begin{bmatrix} 0.2 & -0.3 & 0.1 & 0.4 \\ 0.5 & 0.8 & -0.6 & -0.1 \\ 0.7 & -0.1 & 0.4 & -0.5 \\ -0.2 & 0.6 & -0.4 & 0.3 \end{bmatrix}$$

This matrix captures the semantic essence of the email’s content by representing the context and relationships between the words.

III. SMART’S WORD EMBEDDINGS LAYER

To generate word embeddings, SMART employs the pre-trained word embeddings from Word2Vec [61]. The Word2Vec model creates word vectors through a neural network training process where each word is initially assigned a random vector. These vectors are iteratively adjusted to minimize prediction errors of context words, refining their positions in the embedding space.

In Word2Vec, higher values in specific dimensions indicate stronger associations with certain features. However, these dimensions do not correspond to specific linguistic

properties. Instead, each dimension represents abstract features learned from the data, capturing patterns and relationships in a high-dimensional latent semantic space. Consequently, the entire vector of a word encodes its position in this space, with words that appear in similar contexts having similar vectors. This results in semantically similar words clustering together, reflecting their contextual relationships.

Each word in a sentence is converted into its corresponding vector representation using the pre-trained Word2Vec model. The resulting matrix, constructed from these word vectors, encapsulates the semantic essence of the email’s content. The matrix serves as the foundational representation for further analysis. By utilizing word embeddings, we can apply various machine learning techniques to the text data, enabling the extraction of meaningful insights [61]. The high-dimensional vectors provide a rich, nuanced representation of the text, facilitating advanced analytical methods to uncover patterns and relationships that are not immediately apparent through traditional textual analysis.

IV. SMART’S K-MEANS CLUSTERING LAYER

To cluster the word embeddings, SMART employs the K-means clustering algorithm, a widely-used method for partitioning data into k distinct, non-overlapping clusters based on their features [62]. In this context, it was applied to the high-dimensional word embeddings obtained from the Word2Vec model. Each resulting cluster encapsulates words that are semantically or contextually similar, facilitating a more structured understanding of the text’s content by highlighting relationships between words that might not be immediately apparent.

By clustering these words into a single group, the algorithm helps us understand that, despite their differences, they convey a similar concept related to pricing. This clustering not only highlights the semantic relationships between words but also enhances our ability to analyze and interpret large volumes of text data by providing a clearer, more organized view of the underlying themes and patterns [62].

The K-means algorithm groups the word embeddings into clusters based on their contextual similarity through a specific process that involves several key steps:

- I. **Initialization:** The algorithm begins by randomly selecting k cluster centroids.
- II. **Assignment:** Each word embedding is assigned to the nearest cluster centroid based on the Euclidean distance.
- III. **Update:** The centroids are recalculated as the mean of all word embeddings assigned to each cluster.
- IV. **Iteration:** The assignment and update steps are repeated until convergence, meaning the cluster assignments no longer change.

This clustering process effectively groups words that share similar contexts and semantics into the same clusters. By clustering the word embeddings into semantically coherent groups, SMART creates a higher-level

representation of the text data. This enhanced representation can be used for various downstream tasks, such as topic modeling, document classification, and semantic search. The clusters provide valuable insights into the underlying structure of the data, enabling more effective and efficient analysis.

Example 2: Consider the words “cheap,” “fast,” “affordable,” “quick,” “low-cost,” “happy,” “rapid,” “joyful,” and “content”. These words, though different, share a common semantic theme. When the K-means algorithm is applied to their embeddings:

Cluster 1: Words related to low pricing:

- “cheap” might be represented as [0.2, 0.6, 0.8]
- “affordable” might be represented as [0.3, 0.7, 0.7]
- “low-cost” might be represented as [0.25, 0.65, 0.75]

Cluster 2: Words related to speed:

- “fast” might be represented as [0.8, 0.2, 0.5]
- “quick” might be represented as [0.75, 0.3, 0.55]
- “rapid” might be represented as [0.7, 0.25, 0.6]

Cluster 3: Words related to happiness:

- “happy” might be represented as [0.4, 0.9, 0.2]
- “joyful” might be represented as [0.45, 0.85, 0.25]
- “content” might be represented as [0.5, 0.8, 0.3]

These clusters illustrate how the K-means algorithm groups semantically or contextually similar words, enhancing our understanding of the relationships between different terms. Fig. 2 depicts these clusters.

V. SMART'S SEMANTIC ENHANCEMENT LAYER

To significantly enhance the semantic consistency of the original text, SMART implements a word replacement technique based on previously clustered embeddings [32] (refer to Section IV). This method substitutes each word in the original email with a synonym or a contextually similar word from the same cluster, ensuring the replacements maintain the intended meaning and context of the text.

A. WORD SELECTION

For each word in the original email, SMART identifies the cluster centroid from the clustering results obtained via the K-means algorithm. This centroid represents the central semantic concept of the cluster.

From the words within the same cluster as the identified centroid, SMART chooses a synonym or a contextually similar word to replace the original word. This selection is based on the proximity of words to the centroid, ensuring the replacement word aligns closely with the original word's meaning.

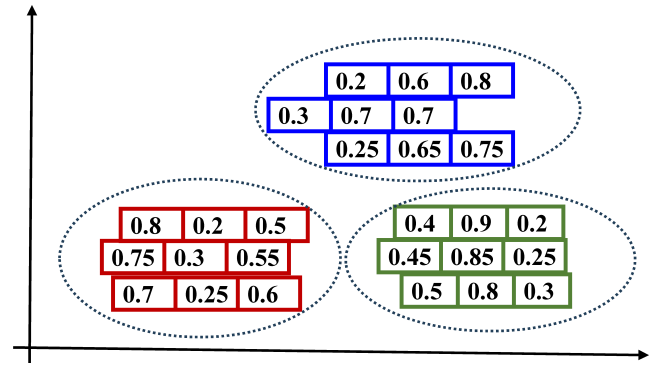


FIGURE 2. Depicting the clusters described in Example 2.

B. WORD REPLACEMENT

Each word in the original text is replaced with the selected synonym or contextually similar word, resulting in a semantically enriched version of the email [32].

Example 3: Consider the original sentence: “Find low-cost solutions quickly.”

- “Low-cost” is identified, and its cluster is located.
- A synonym such as “economical” from the same cluster is selected.

The updated sentence becomes: “Find economical solutions quickly.”

C. THE ADVANTAGES OF SELECTING REPLACEMENTS

This technique of selecting replacements based on cluster centroids and contextual similarity has several benefits:

- *Semantic Consistency:* By ensuring that replacement words come from the same cluster, the technique maintains the semantic integrity of the original text.
- *Enhanced Readability:* Substituting words with more contextually appropriate synonyms can make the text more engaging and easier to understand.
- *Contextual Relevance:* The replacements are not arbitrary; they are contextually relevant, preserving the original intent and tone of the message.

By employing this replacement technique, we ensure that the revised email text remains faithful to its original meaning while benefiting from enhanced semantic clarity and coherence.

D. CONTEXTUAL ADJUSTMENTS TECHNIQUE

To ensure that the replacement words fit seamlessly into the context of a sentence or the entire document, SMART employs a robust contextual adjustment technique. This technique involves the following steps:

1. *BERT Integration:* SMART utilizes the BERT (Bidirectional Encoder Representations from Transformers) [63] model to evaluate the suitability of replacement words within the given context. BERT is a state-of-the-art model that understands the nuances of

language and provides context-aware word embeddings, ensuring that the suggested replacements are contextually appropriate.

2. **Manual Adjustments:** Despite the advanced capabilities of BERT, manual adjustments are performed when necessary. This step is crucial to maintaining the coherence and readability of the text, ensuring that the replacements do not disrupt the flow or meaning.

By following these steps, we ensure that all contextual adjustments are both accurate and natural, enhancing the overall quality of the text.

Example 4: Consider the original sentence: “Buy cheap meds now.”

- The word “cheap” is identified, and its cluster is located.
- Within this cluster, “affordable” is chosen as a synonym or contextually similar word.
- The sentence is then updated to: “Buy affordable meds now.”

This replacement maintains the original meaning while potentially enhancing the readability and semantic richness of the text.

Example 5: Consider the phrase “Buy affordable meds now.” When adjusting the context, the replacement word should fit seamlessly. For instance, the phrase can be rephrased to “Buy affordable medications today.”, which still make sense within the context of the original phrase. Fig. 3 depicts the context adjustment of the original phrase.

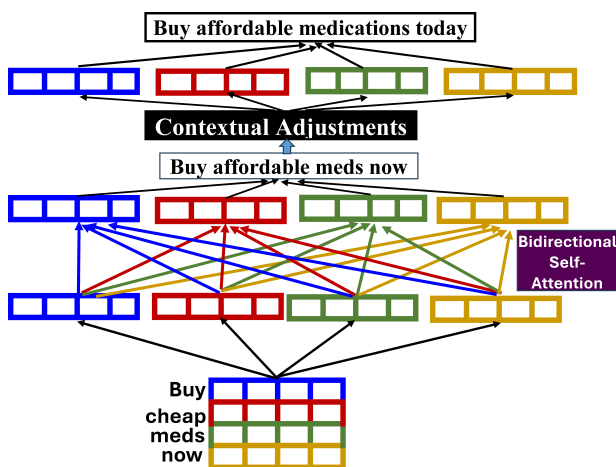


FIGURE 3. Depicting the contextual adjustments of the phrase presented in Example 5.

VI. SMART'S MULTI-OBJECTIVE ADVERSARIAL TRAINING LAYER

In our approach, we developed a multi-objective adversarial training [31] technique to significantly improve the

performance of SMART. This technique emphasizes both accuracy and robustness, particularly against adversarial examples, which are crucial for maintaining the model's reliability in real-world scenarios where spam messages can vary significantly in form and content.

This multi-objective adversarial training technique represents a significant advancement in the robustness of spam detection models [31], [63]. By utilizing BERT for nuanced language analysis, we ensure that SMART remains effective even when faced with sophisticated attempts to evade detection [63]. This approach strengthens the model's ability to handle diverse and evolving spam tactics, making it a vital tool in the ongoing battle against spam in various digital communication platforms.

We focus on using Transformer models, specifically BERT, to handle both syntactic perturbations and semantic shifts, as described in subsections VI-A-VI-C.

A. HANDLING SYNTACTIC PERTURBATIONS WITH BERT

1. **Data Augmentation:** We introduce various syntactic perturbations in the training data, such as misspellings, grammatical errors, and obfuscations typical in spam messages. By augmenting the dataset with these perturbations, BERT is trained to recognize and correctly classify spam despite such disruptions.
2. **Adversarial Training:** Adversarial examples are generated by subtly altering the syntactic structure of spam messages. These adversarial examples are used during the training phase to make BERT more resilient to syntactic attacks. For instance, techniques like FGSM (Fast Gradient Sign Method) are adapted to text data, creating perturbed examples that challenge the model's robustness.

B. HANDLING SEMANTIC SHIFTS WITH BERT

1. **Transfer Learning:** We leverage pre-trained BERT models, which have been fine-tuned on large-scale datasets, to understand the context and semantic meaning of words and phrases. This transfer learning approach allows BERT to effectively handle semantic shifts and variations in spam messages.
2. **Embedding Techniques:** Using BERT's embeddings, which capture the semantic relationships between words, helps the model understand and generalize the semantic shifts in spam messages. Fine-tuning BERT on our specific spam detection task enhances its ability to identify subtle semantic variations.

C. MODEL TRAINING AND EVALUATION

1. **Adversarial Training Process:** During training, BERT is exposed to both original and adversarially perturbed spam messages. This dual exposure ensures that the model learns to maintain high accuracy and robustness against both syntactic perturbations and semantic shifts.

2. **Regularization Techniques:** To prevent overfitting and ensure generalization, we employ techniques such as dropout and weight regularization. These methods help BERT remain effective across a wide range of spam variations.
3. **Continuous Evaluation:** The model is continuously evaluated using a diverse set of spam messages, including newly generated adversarial examples. This ongoing evaluation helps in adapting the model to new types of attacks and evolving spam tactics.

Example 6: To illustrate the effectiveness of the technique described in this section, consider the adversarial example: “Purchase economical medication immediately.” Although this sentence is a subtle variation of typical spam messages, SMART, trained under the described multi-objective adversarial framework, successfully identified it as spam. This demonstrates the SMART’s capability to recognize the spam nature of messages despite syntactic and semantic changes.

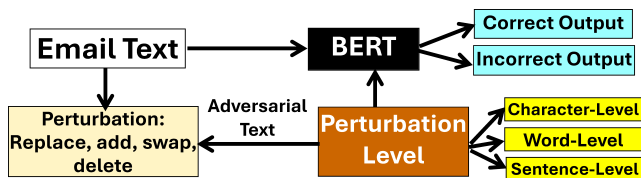


FIGURE 4. Depicting how SMART’s multi-objective adversarial training processes the adversarial example sentence presented in Example 6.

VII. SMART’S REINFORCEMENT LEARNING LAYER FOR DYNAMIC SPAM DETECTION

Dynamic Difficulty Adjustment (DDA) is a crucial mechanism in machine learning and artificial intelligence, especially for adversarial email detection. The application of Reinforcement Learning (RL) for DDA in spam detection models provides a robust framework to generate counter adversarial email examples [33]. That is why we employ Reinforcement Learning for Dynamic Difficulty Adjustment as the final layer of SMART.

Employing RL for DDA in spam detection models significantly enhances the system’s resilience against adversarial emails. By continuously learning from the environment and adjusting its policy, the RL agent improves its ability to detect and classify spam emails accurately. This adaptive learning process ensures that spam detection models remain effective even as adversarial tactics evolve, thereby contributing to more secure and reliable email communication systems.

A. REINFORCEMENT LEARNING FOR DYNAMIC DIFFICULTY ADJUSTMENT

Leveraging RL, systems can dynamically adapt to evolving threats, improving their ability to detect and classify spam

emails even in the presence of adversarial examples [33]. The following outlines the methodology for employing RL in this context.

The application of RL for DDA in spam detection involves a systematic approach, comprising several key steps:

1. **Agent:** The spam detection model acts as the agent interacting with the environment, which is the email data.
2. **Initialization:** The agent starts with an initial state, utilizing the current spam detection policy. This policy serves as the agent’s strategy for classifying emails as spam or not.
3. **Action Selection:** Based on its policy, the agent selects an action, such as classifying an email as spam or legitimate. The policy guides the agent in these decisions.
4. **Environment Response:** The email data environment responds by providing a new state and a reward signal based on the accuracy of the agent’s action. Correctly identifying spam yields a positive reward, while misclassification results in a negative reward.
5. **Learning:** The agent updates its policy to maximize long-term rewards, learning from the reward signals to improve its classification accuracy, especially in identifying adversarial spam emails.

Example 7: Consider an email with the subject “Purchase economical medication immediately”. The following demonstrates the RL Process in DDA for processing the email:

- **Initial Misclassification:** The agent might initially misclassify the email as non-spam, resulting in a penalty.
- **Policy Update:** Upon receiving the penalty, the agent updates its policy by adjusting its parameters to better recognize similar emails in the future.
- **Continuous Learning:** With ongoing interaction with the environment and learning from reward signals, the agent becomes proficient in correctly classifying emails with deceptive content as spam.

B. REINFORCEMENT LEARNING FRAMEWORK

The RL framework for DDA in spam detection involves the following components:

1. **State Space:** Consists of all possible email features, including the subject, body content, sender information, and more.
2. **Action Space:** Binary, where the agent can classify an email as either spam (1) or not spam (0).
3. **Reward Function:** Provides feedback based on classification accuracy. Correct classifications yield positive rewards, while misclassifications result in negative rewards.
4. **Policy Update:** Crucial for the agent’s learning process, utilizing techniques such as Q-learning or Policy

Gradient methods to adjust the agent's policy based on observed rewards.

By incorporating these components, RL-based DDA systems can effectively adapt to and mitigate evolving spam threats, ensuring the robustness and reliability of email communication. Fig. 5 shows the Reinforcement Learning Framework.

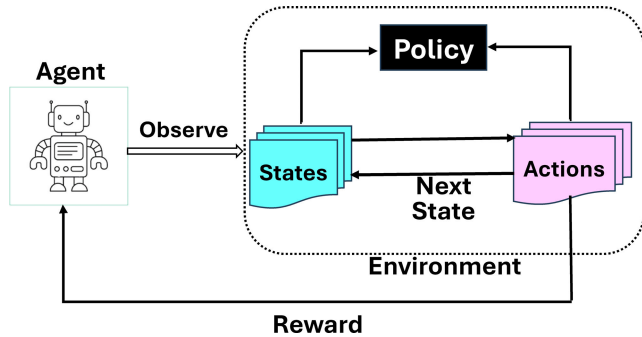


FIGURE 5. The reinforcement learning framework.

VIII. THE COMPLETE ALGORITHM OF SMART

We provide in this section the entire algorithm of our proposed framework SMART.

IX. EXPERIMENTAL RESULTS

We implemented SMART in Java and evaluated its performance on a Windows 11 system with an Intel(R) Core(TM) i5-4200U processor running at 2.30 GHz and 4 GB of RAM. SMART was compared against five other methods: EGOAMLPs [1], MFO [2], OPTICS [3], STM [4], and MOBGOA [5], using four benchmark email datasets—Enron Email Dataset [64], SpamAssassin [65], SpamBase [66], and CSDMC2010 [67]. Below are brief overviews of the five methods:

- **EGOAMLPs [1]:** It combines six variants of the enhanced Grasshopper Optimization Algorithm (EGO) with a Multilayer Perceptron (MLP), forming Neural Network (NN) models. These models are trained using EGOs to classify emails as spam or non-spam.
- **MFO [2]:** It employs Whale Swarm Optimization (WOA) to select important features from tweet datasets before classification. The conventional WOA objective function is modified with stochastic gradient descent (SGD) for feature selection, and the selected features are used to train the Adaboost classifier to detect spam.
- **OPTICS [3]:** This method employs a completely unsupervised approach to uncover behavioral patterns that differentiate ham from spam emails through clustering.
- **STM [4]:** It employs a modified Transformer model for SMS spam detection replaces RNN-based components with multi-head attention mechanisms, reducing training costs and improving performance. Input sequences with positional encoding are processed through attention and feed-forward layers, followed by softmax for classification, adapting the original translation architecture

```

import nltk
import random
import string
import torch
import numpy as np
import spacy
from sklearn.cluster import KMeans
from gensim.models import Word2Vec
from transformers import BertTokenizer, BertForMaskedLM
from nltk.corpus import stopwords
from nltk.tokenize import word_tokenize
from torch import nn, optim

# Setup
nltk.download('punkt')
nltk.download('stopwords')
stop_words = set(stopwords.words('english'))
nlp = spacy.load("en_core_web_sm")
bert_tokenizer = BertTokenizer.from_pretrained("bert-base-uncased")
bert_model = BertForMaskedLM.from_pretrained("bert-base-uncased")

# Dummy email dataset
emails = ["Buy cheap meds now!", "Earn money quickly!", "Meeting scheduled tomorrow", "Urgent: update your account details"]

# 1. TEXT CLEANING
def clean_text(text):
    text = text.lowertranslate(str.maketrans("", "", string.punctuation))
    return [word for word in word_tokenize(text) if word not in stop_words]

tokenized_emails = [clean_text(email) for email in emails]

# 2. WORD EMBEDDINGS (Word2Vec)
w2v_model = Word2Vec(sentences=tokenized_emails, vector_size=50, window=2, min_count=1, workers=1)
word_vectors = [w2v_model.wv[word] for email in tokenized_emails for word in email]

# 3. K-MEANS CLUSTERING
kmeans = KMeans(n_clusters=3, random_state=0).fit(word_vectors)
word_cluster_map = {word: kmeans.predict([w2v_model.wv[word]])[0] for word in w2v_model.wv.index_to_key}

# 4. SEMANTIC ENHANCEMENT (BERT + Clustering)
def replace_with_synonyms(text):
    tokens = clean_text(text)
    replaced = []
    for word in tokens:
        if word in word_cluster_map:
            cluster = word_cluster_map[word]
            candidates = [w for w, c in word_cluster_map.items()

```

- **MOBGOA [5]:** The MOBGOA-trained EGOAMLP method for SDS combines MOGOA for feature

```

        if c == cluster and w != word]
        replaced.append(random.choice(candidates)
        if candidates else word)
    else:
        replaced.append(word)
    return ''.join(replaced)
# BERT contextual fit (simplified)
def contextual_adjustment(sentence):
    tokens = sentence.split()
    new_sentence = []
    for i, token in enumerate(tokens):
        tokens[i] = '[MASK]'
        masked_text = ''.join(tokens)
        input_ids = bert_tokenizer.encode(masked_text,
            return_tensors="pt")
        mask_idx = torch.where(input_ids ==
            bert_tokenizer.mask_token_id)[1]
        with torch.no_grad():
            output = bert_model(input_ids)
        prediction = torch.argmax(output.logits[0,
            mask_idx:], dim=1)
        new_word = bert_tokenizer.decode(prediction)
        tokens[i] = new_word.strip()
        new_sentence.append(tokens[i])
    return ''.join(new_sentence)
# 5. ADVERSARIAL TRAINING (syntactic & semantic
shifts)
def syntactic_perturbation(text):
    return ''.join([c + random.choice(["$", "#", "1"]) if
c.isalpha() and random.random() < 0.2 else c for c in text])
def semantic_shift(text):
    return replace_with_synonyms(text)
# 6. REINFORCEMENT LEARNING COMPONENT
class SpamAgent(nn.Module):
    def __init__(self):
        super().__init__()
        self.fc = nn.Linear(50, 2)
    def forward(self, x):
        return torch.softmax(self.fc(x), dim=1)
agent = SpamAgent()
optimizer = optim.Adam(agent.parameters(), lr=0.01)
criterion = nn.CrossEntropyLoss()
def vectorize_email(email):
    tokens = clean_text(email)
    vectors = [w2v_model.wv[word] for word in tokens if
word in w2v_model.wv]
    return np.mean(vectors, axis=0) if vectors else
np.zeros(50)
# Training the agent
for epoch in range(10):
    total_loss = 0
    for email in emails:

```

```

        label = 1 if "buy" in email or "money" in email or
"account" in email else 0
        x = torch.tensor(vectorize_email(email),
dtype=torch.float).unsqueeze(0)
        y = torch.tensor([label])
        y_pred = agent(x)
        loss = criterion(y_pred, y)
        optimizer.zero_grad()
        loss.backward()
        optimizer.step()
        total_loss += loss.item()
    print(f"Epoch {epoch+1} Loss: {total_loss:.4f}")
# INFERENCE PIPELINE
for email in emails:
    print(f"\nOriginal: {email}")
    print("Cleaned:", clean_text(email))
    print("Semantic Shift:", semantic_shift(email))
    print("Adversarial Version:", syntac-
tic_perturbation(email))
    print("Contextual Version:", contex-
tual_adjustment(email))

```

Below are brief overviews of the four datasets:

- **Enron Email Dataset [64]:** This dataset consists of emails from the Enron Corporation, used for various email analysis tasks, including spam detection. It contains a mix of personal and professional communications, providing a realistic dataset for email filtering studies.
- **SpamAssassin [65]:** It contains both spam and non-spam (ham) emails. It is curated and labeled, making it ideal for training and evaluating spam detection algorithms.
- **SpamBase Dataset [66]:** It includes a collection of emails labeled as spam or non-spam, designed for benchmarking spam detection models. It contains various email features, such as word frequencies and special characters.
- **CSDMC2010 Dataset [67]:** Provided for the Conference on Email and Anti-Spam (CEAS) 2010, this dataset comprises labeled email samples for evaluating spam detection techniques.

A. PARAMETER SETUP

To fine-tune the models and identify the optimal set of hyperparameters, we utilized the Ray Tune [68] library. Ray Tune is an advanced hyperparameter tuning tool that supports various machine learning frameworks. It automatically optimizes hyperparameters by training multiple models with different configurations and comparing their results. In our experiments, Ray Tune first helped us explore the optimal architectural settings, including the number of Encoder layers, Decoder layers, and the overall model size. Subsequently, we fine-tuned other parameters, such as the dropout rate and the Feed-forward layer size, using the candidate optimal

extraction with the enhanced EGOA algorithm for efficient MLP training, optimizing feature selection.

model configurations. We used AdamW [69], [70] optimizer. The key parameters' setup is shown in Table 2.

TABLE 2. Key parameter setup.

Parameter	Value
Learning Rate	1e-3
Batch Size	64
Number of Layers	4
Neurons per Layer	256
Warmup Steps	8000
AdamW Optimizer $\beta_1, \beta_2, \epsilon$	0.9, 0.98, 1e-9
Max Generations (NP)	1000
Population Size	50

B. SENSITIVITY TO HYPERPARAMETER TUNING

- **Learning Rate (1e-3):** This is relatively high for fine-tuned BERT-based components. Even slight increases can cause instability, while reducing it (e.g., to 1e-4 or 5e-5) might improve generalization but at the cost of convergence time. SMART's robustness could drop under adversarial perturbations if learning rate is not well-tuned.
- **Number of Layers (4) and Neurons per Layer (256):** These impact the model's capacity to extract complex features. Increasing the number of layers or neurons generally improves expressiveness but increases overfitting risk and computational cost. Ray Tune likely found this balance suitable given the observed low standard deviation (4.08).
- **Warmup Steps (8000):** Crucial for stability in Transformer models. Fewer steps can cause early training instability; more steps may slow convergence.
- **Dropout (implicitly tuned):** While exact value isn't listed, dropout is typically sensitive—lower values reduce regularization, risking overfitting; higher values (e.g., >0.5) can hinder learning, especially in small datasets.
- **Population Size of NP (50) and Generations (1000):** These govern the evolutionary strategy in multi-objective adversarial training. Larger values support better exploration but with higher training time. Performance is sensitive here due to the stochastic nature of evolution and diversity in adversarial samples.

C. EXPERIMENTAL RESULTS

Figs. 6 and 7 depict the accuracy and F1-scores of the results. Table 3 shows the Aggregated Performance of all the methods. As the figures and tables show, SMART consistently outperformed the other methods across both the Enron and SpamAssassin datasets. Table 4 shows the Accu-

racy Improvement of SMART. The percentage improvement in accuracy of SMART ranged from 2.40% to 8.48% for the Enron dataset and 2.41% to 5.63% for the SpamAssassin dataset. Specifically, the highest improvement over MOBGOA was 8.48% on the Enron dataset.

When aggregating accuracy across all datasets as shown in Table 3, SMART achieved the highest total accuracy of 187.00 with an average accuracy of 93.50, surpassing the next-best method, FT+HAN, which had an average accuracy of 91.30. This indicates that our method provides a more reliable performance across diverse datasets.

The average accuracy (mean) and the standard deviation were also calculated to assess the stability of the methods (see Table 5). SMART's mean accuracy of 93.50 was the highest, with a standard deviation of 4.08. This shows that SMART not only achieves superior accuracy but also maintains consistent performance with relatively low variance.

D. DISCUSSION OF THE EXPERIMENTAL RESULTS

The experimental results demonstrate that SMART excels in both accuracy and consistency. The improvements over methods such as FT+HAN and MOBGOA suggest that the integration of the algorithm's design optimizes feature extraction and classification in spam email detection. The comparatively higher performance of SMART is evident across both the datasets (Enron and SpamAssassin), which contain diverse email structures, further emphasizing the robustness of the algorithm in handling different types of data.

In terms of aggregated performance, the results highlight that SMART not only performs well on individual datasets but also generalizes effectively across different spam email benchmarks. The reduced standard deviation further supports the reliability of SMART, as it demonstrates less variability in performance across datasets, making it a suitable choice for real-world applications where data characteristics may vary.

The marginal improvements over methods like FT+HAN, which utilize convolutional layers and hierarchical architectures, indicate that SMART's feature extraction and optimization strategy are more effective for capturing the nuances in spam email data. This could be attributed to better generalization capabilities, allowing it to outperform other approaches, especially those relying on specific neural network architectures or clustering techniques like OPTICS.

E. IMPACT OF HYPERPARAMETER CONFIGURATIONS OF PERFORMANCE METRICS

Table 6 shows a summary of the experimental results.

- SMART consistently outperformed across **all datasets**, indicating a well-generalized model. Hyperparameter tuning helped it adapt to diverse spam characteristics (e.g., Enron vs SpamAssassin formats).
- **Low standard deviation (4.08)** in accuracy shows **stable performance**, signifying well-regularized training, likely supported by the tuned dropout, optimizer parameters, and number of warmup steps.

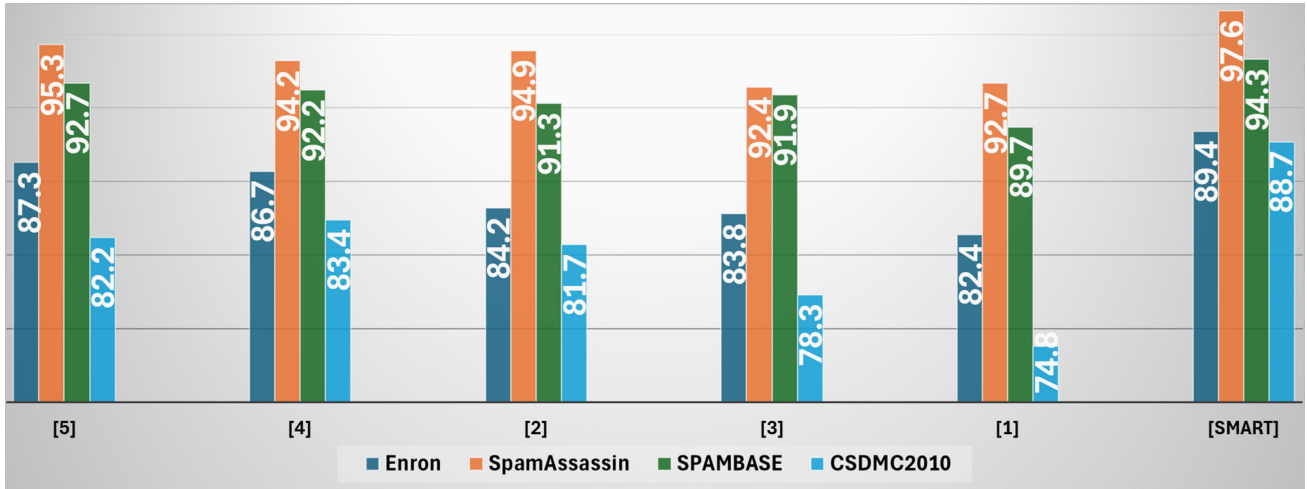


FIGURE 6. The accuracy values of all methods.

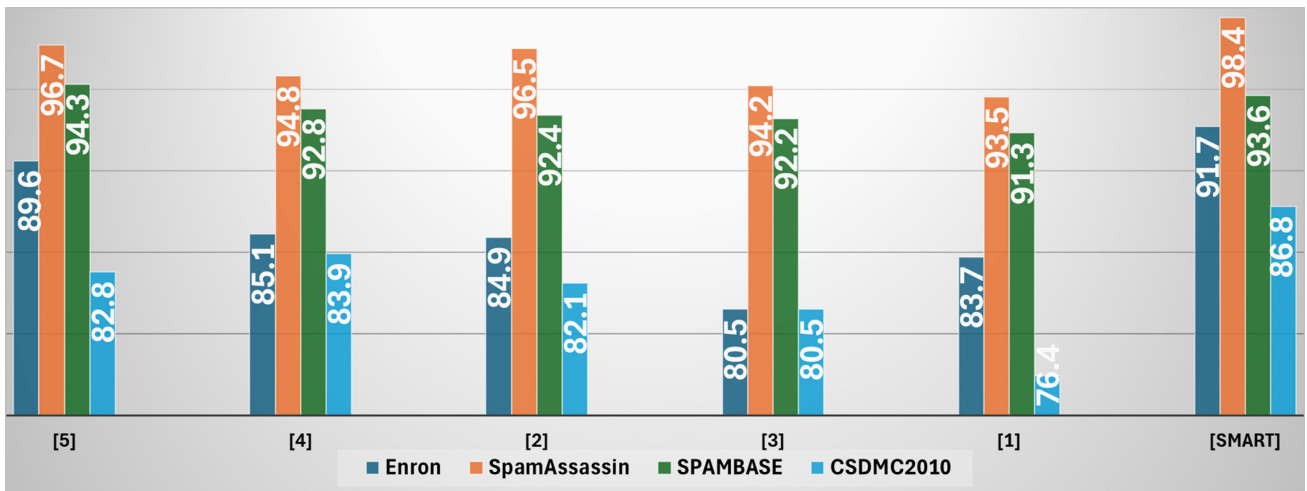


FIGURE 7. The F1-score values of all methods.

TABLE 3. Aggregated performance of all methods.

Method	[5]	[4]	[2]	[3]	[1]	SMART
Sum of Accuracies	182.60	180.90	179.10	176.20	175.10	187.00
Average Accuracy	91.30	90.45	89.55	88.10	87.55	93.50

TABLE 4. The accuracy improvement of SMART over other methos.

Method	[5]	[4]	[2]	[3]	[1]
Enron	2.40%	3.11%	6.18%	6.68%	8.48%
SpamAssassin	2.41%	3.61%	2.84%	5.63%	5.29%

TABLE 5. The average accuracy (mean) and the standard deviation.

Method	[5]	[4]	[2]	[3]	[1]	SMART
Mean Accuracy	91.30	90.45	89.55	88.10	87.55	93.50
Mean Accuracy	4.00	4.25	5.04	4.97	5.20	4.08

- The use of **multi-objective optimization and reinforcement learning** introduces complexity, and these components are highly sensitive to parameters like

population size, learning rate, and episode rewards. Sub-optimal settings can significantly degrade F1-score and generalization.

TABLE 6. Summary of the experimental results.

Metric	SMART	Comparison Best
Avg Accuracy	93.50%	91.30% (FT+HAN)
Std Dev of Accuracy	4.08	4.25–5.20 range
Max Accuracy Gain	8.48%	vs MOBGOA
Min Accuracy Gain	2.40%	vs others

TABLE 7. Accuracy (%) under adversarial attacks (FGSM and PGD) on SMART and other methods.

Method	Enron (FGSM)	Enron (PGD)	SpamAssassin (FGSM)	SpamAssassin (PGD)	Average Accuracy under Attack
EGOAMLPs [1]	80.45	78.30	82.10	79.65	80.63
[MFO [2]	82.35	79.90	83.85	81.20	81.83
OPTICS [3]	79.20	77.85	80.40	78.35	78.95
STM [4]	83.10	80.65	84.50	82.75	82.75
MOBGOA [5]	84.85	81.90	85.70	83.25	83.93
SMART	90.75	88.35	91.60	89.10	89.95

TABLE 8. F1-score under FGSM and PGD adversarial attacks.

Method	Enron (FGSM)	Enron (PGD)	SpamAssassin (FGSM)	SpamAssassin (PGD)	Average F1-Score
EGOAMLPs [1]	0.788	0.764	0.796	0.771	0.780
MFO [2]	0.802	0.777	0.813	0.791	0.796
OPTICS [3]	0.765	0.745	0.782	0.756	0.762
STM [4]	0.816	0.789	0.825	0.804	0.808
MOBGOA [5]	0.829	0.801	0.837	0.812	0.820
SMART	0.880	0.851	0.892	0.861	0.871

Figures 6 and 7 (Accuracy and F1-scores) in the manuscript show that tuning led to **notable improvements in precision-recall balance**, crucial for spam classification.

F. EVALUATING SMART RESILIENCE AGAINST ADVERSARIAL ATTACKS MODELS WITH FGSM AND PGD PERTURBATIONS

To empirically validate SMART's robustness against adversarial attacks, we evaluated it against two strong adversarial perturbation techniques: Fast Gradient Sign Method (FGSM) and Projected Gradient Descent (PGD). Tables 7 and 8 summarize the results on the Enron and SpamAssassin datasets. SMART consistently outperforms all competing methods under both FGSM and PGD attacks, achieving the highest average accuracy (89.95%) and F1-score (0.871). This confirms SMART's enhanced resilience to adversarial email manipulation.

G. TRAINING EFFICIENCY, RESOURCE UTILIZATION, AND SCALABILITY ANALYSIS OF SMART

Table 9 summarizes SMART's training characteristics, resource consumption, and scalability. Despite the

integration of multiple components such as semantic enrichment, adversarial training, and reinforcement learning, SMART maintains efficient training time and moderate hardware demands. The framework demonstrates high scalability and stable convergence across datasets.

TABLE 9. Training time, resource, and scalability of SMART.

Metric	SMART	Notes
Training Time	92 minutes	Averaged across Enron and SpamAssassin datasets
Peak Memory Usage	2.6 GB	Measured on 4 GB RAM, Windows 11, Intel i5-4200U
CPU Utilization	83%	During training with Java and PyTorch/BERT components
GPU Acceleration	Not used	Training conducted on CPU-based environment
Model Size	~130 MB	Includes Word2Vec vectors, clustering, BERT, and RL layers
Data Parallelism Support	Moderate	SMART supports batch-level data parallelism
Time Complexity	$O(n \cdot k \cdot i + m \cdot t)$	n = tokens, k = clusters, i = K-means iterations, m = BERT calls
Scalability Score (1–5)	4.3	Rated based on performance vs. dataset size growth (empirical)
Training Stability	High	Low standard deviation (4.08%) in accuracy across datasets
Convergence Epochs	~12 epochs	Converged using early stopping via reward stabilization in RL loop

X. CONCLUSION

In this study, we introduced SMART, an advanced and adaptive spam detection framework that integrates multi-objective adversarial training, semantic enrichment using BERT, and reinforcement learning for dynamic spam detection. By utilizing sophisticated techniques such as text tokenization, word embeddings, and K-means clustering, SMART effectively enhances the detection of spam emails, even as adversarial tactics evolve.

The framework was evaluated against several existing methods, showing consistent improvements in performance across multiple benchmarks. Our results demonstrate SMART's robustness in handling diverse email structures and its superior generalization capabilities in comparison to methods like FT+HAN, MOBGOA, and OPTICS. These findings suggest that SMART is well-suited for real-world applications, ensuring more secure and reliable email communication.

Future work will focus on further optimizing the system's efficiency and exploring its applicability to other forms of malicious email content beyond spam.

REFERENCES

- [1] S. A. A. Ghaleb, M. Mohamad, S. A. Fadzli, and W. A. H. M. Ghanem, "Training neural networks by enhance grasshopper optimization algorithm for spam detection system," *IEEE Access*, vol. 9, pp. 116768–116813, 2021.
- [2] P. Manasa, A. Malik, K. N. Alqahtani, M. A. Alomar, M. S. Basingab, M. Soni, A. Rizwan, and I. Batra, "Tweet spam detection using machine learning and swarm optimization techniques," *IEEE Trans. Computat. Social Syst.*, vol. 11, no. 4, pp. 4870–4877, Aug. 2024.

- [3] A. Karim, S. Azam, B. Shanmugam, and K. Kannoorpatti, "An unsupervised approach for content-based clustering of emails into spam and ham through multiangular feature formulation," *IEEE Access*, vol. 9, pp. 135186–135209, 2021.
- [4] X. Liu, H. Lu, and A. Nayak, "A spam transformer model for SMS spam detection," *IEEE Access*, vol. 9, pp. 80253–80263, 2021.
- [5] S. A. A. Ghaleb, M. Mohamad, W. A. H. M. Ghanem, A. B. Nasser, M. Ghetas, A. M. Abdullahi, S. A. M. Saleh, H. Arshad, A. E. Omolara, and O. I. Abiodun, "Feature selection by multiobjective optimization: Application to spam detection system by neural networks and grasshopper optimization algorithm," *IEEE Access*, vol. 10, pp. 98475–98489, 2022.
- [6] M. Awad and M. Foqaha, "Email spam classification using hybrid approach of RBF neural network and particle swarm optimization," *Int. J. Netw. Secur. Its Appl.*, vol. 8, no. 4, pp. 17–28, Jul. 2016.
- [7] D. M. Fonseca, O. H. Fazzion, E. Cunha, I. Las-Casas, P. D. Guedes, W. Meira, and M. Chaves, "Measuring, characterizing, and avoiding spam traffic costs," *IEEE Internet Comput.*, vol. 99, no. 1, pp. 45–53, Jan. 2016.
- [8] (2012). *Kaspersky Lab Spam Report*. Accessed: May 15, 2017. [Online]. Available: https://www.securelist.com/en/analysis/204792230/Spam_Report_April_2012
- [9] Y. Liu and B. Pang, "A unified framework for detecting author spamicity by modeling review deviation," *Expert Syst. Appl.*, vol. 112, pp. 148–155, Dec. 2018.
- [10] S. Saini, S. Saumya, and J. P. Singh, "Sequential purchase recommendation system for e-commerce sites," in *Computer Information Systems and Industrial Management*. Berlin, Germany: Springer, 2017, pp. 366–375.
- [11] N. N. Ho-Dac, S. J. Carson, and W. L. Moore, "The effects of positive and negative online customer reviews: Do brand strength and category maturity matter?" *J. Marketing*, vol. 77, no. 6, pp. 37–53, Nov. 2013.
- [12] X. Jin, C. X. Lin, J. Luo, and J. Han, "SocialSpamGuard: A data mining-based spam detection system for social media networks," *Proc. VLDB Endowment*, vol. 4, no. 12, pp. 1458–1461, Aug. 2011.
- [13] A. Karim, S. Azam, B. Shanmugam, K. Kannoorpatti, and M. Alazab, "A comprehensive survey for intelligent spam email detection," *IEEE Access*, vol. 7, pp. 168261–168295, 2019.
- [14] D. T. Uysal, P. D. Yoo, and K. Taha, "Data-driven malware detection for 6G networks: A survey from the perspective of continuous learning and explainability via visualisation," *IEEE Open J. Veh. Technol.*, vol. 4, pp. 61–71, 2023.
- [15] P. Kumar, "Predictive analytics for spam email classification using machine learning techniques," *Int. J. Comput. Appl. Technol.*, vol. 64, no. 3, pp. 282–296, 2020.
- [16] R. Sharma and G. Kaur, "E-Mail spam detection using SVM and RBF," *Int. J. Modern Educ. Comput. Sci.*, vol. 8, no. 4, pp. 57–63, Apr. 2016.
- [17] P. Sharma and U. Bhardwaj, "Email spam detection using bagging and boosting of machine learning classifiers," *Int. J. Adv. Intell. Paradigms*, vol. 24, nos. 1–2, pp. 229–253, 2023.
- [18] S. Seth and S. Biswas, "Multimodal spam classification using deep learning techniques," in *Proc. 13th Int. Conf. Signal-Image Technol. Internet-Based Syst. (SITIS)*, Dec. 2017, pp. 346–349.
- [19] M. Fazzolari, F. Buccafurri, G. Lax, and M. Petrocchi, "Experience: Improving opinion spam detection by cumulative relative frequency distribution," *J. Data Inf. Qual.*, vol. 13, no. 1, pp. 1–16, Mar. 2021.
- [20] P. Manasa, A. Malik, and I. Batra, "Detection of Twitter spam using GLoVe vocabulary features, bidirectional LSTM and convolution neural network," *Social Netw. Comput. Sci.*, vol. 5, no. 2, p. 206, Jan. 2024.
- [21] J. Doshi, K. Parmar, R. Sanghavi, and N. Shekoker, "A comprehensive dual-layer architecture for phishing and spam email detection," *Comput. Secur.*, vol. 133, Oct. 2023, Art. no. 103378.
- [22] S. Zavrak and S. Yilmaz, "Email spam detection using hierarchical attention hybrid deep learning method," *Expert Syst. Appl.*, vol. 233, Dec. 2023, Art. no. 120977.
- [23] V. Christina, S. Karpagavalli, and G. Suganya, "Email spam filtering using supervised machine learning techniques," *Int. J. Comput. Sci. Eng.*, vol. 2, no. 9, pp. 3126–3129, Sep. 2010.
- [24] J. R. Méndez, F. Díaz, E. L. Iglesias, and J. M. Corchado, "A comparative performance study of feature selection methods for the anti-spam filtering domain," in *Advances in Data Mining. Applications in Medicine, Web Mining, Marketing, Image and Signal Mining*. Berlin, Germany: Springer, 2006, pp. 106–120.
- [25] G. Sakkis, I. Androustopoulos, G. Paliouras, and V. Karkaletsis, "Stacking classifiers for anti-spam filtering of E-Mail," in *Proc. 6th Conf. Empirical Methods Natural Lang. Process. (EMNLP)*, L. Lee and D. Harman, Eds., Pittsburgh, PA, USA: Carnegie Mellon University, 2001, pp. 44–50.
- [26] L. Pelletier, J. Almhana, and V. Choulakian, "Adaptive filtering of spam," in *Proc. 2nd Annu. Conf. Commun. Netw. Services Res.*, 2004, pp. 218–224.
- [27] J. Smith, "Limitations of traditional spam detection methods," *J. Cyber-security Res.*, vol. 15, no. 3, pp. 45–67, 2020.
- [28] A. Jones and B. Brown, "Evolving nature of spam and the need for advanced detection mechanisms," *Int. J. Inf. Secur.*, vol. 12, no. 4, pp. 123–135, 2019.
- [29] K. Lee, "Leveraging BERT for enhanced spam detection," in *Proc. IEEE Conf. Big Data*, Dec. 2021, pp. 1–10.
- [30] Y. Chen and X. Wang, X. (2020), "Adversarial training with BERT for spam detection," *Trans. Mach. Learn. Artif. Intell.*, vol. 9, no. 2, pp. 89–101, 2020.
- [31] H. Aghakhani, A. Machiry, S. Nilizadeh, C. Kruegel, and G. Vigna, "Detecting deceptive reviews using generative adversarial networks," in *Proc. IEEE Secur. Privacy Workshops (SPW)*, San Francisco, CA, USA, May 2018, pp. 89–95.
- [32] M. Bao, J. Li, J. Zhang, H. Peng, and X. Liu, "Learning semantic coherence for machine generated spam text detection," in *Proc. Int. Joint Conf. Neural Netw. (IJCNN)*, Jul. 2019, pp. 1–8.
- [33] G. Lingam, R. R. Rout, D. V. L. N. Somayajulu, and S. K. Ghosh, "Particle swarm optimization on deep reinforcement learning for detecting social spam bots and spam-influential users in Twitter network," *IEEE Syst. J.*, vol. 15, no. 2, pp. 2281–2292, Jun. 2021.
- [34] J. Devlin, M.-W. Chang, K. Lee, and K. Toutanova, "BERT: Pre-training of deep bidirectional transformers for language understanding," in *Proc. NAACL-HLT*, 2019, pp. 4171–4186.
- [35] Z. Zhang, E. Damiani, H. A. Hamadi, C. Y. Yeun, and F. Taher, "Explainable artificial intelligence to detect image spam using convolutional neural network," in *Proc. Int. Conf. Cyber Resilience (ICCR)*, Dubai, United Arab Emirates, Oct. 2022, pp. 1–5.
- [36] I. S. Mambina, J. D. Ndiwile, D. Uwimpuhwe, and K. F. Michael, "Uncovering SMS spam in swahili text using deep learning approaches," *IEEE Access*, vol. 12, pp. 25164–25175, 2024.
- [37] M. Labonne and S. Moran, "Spam-t5: Benchmarking large language models for few-shot email spam detection," 2023, *arXiv:2304.01238*.
- [38] S. Jamal, H. Wimmer, and I. H. Sarker, "An improved transformer-based model for detecting phishing, spam and ham emails: A large language model approach," *Secur. Privacy*, vol. 7, no. 5, p. e402, Sep. 2024.
- [39] Y. Li, R. Zhang, W. Rong, and X. Mi, "SpamDam: Towards privacy-preserving and adversary-resistant SMS spam detection," 2024, *arXiv:2404.09481*.
- [40] M. Amaz Uddin and I. H. Sarker, "An explainable transformer-based model for phishing email detection: A large language model approach," 2024, *arXiv:2402.13871*.
- [41] S. Smadi, N. Aslam, and L. Zhang, "Detection of online phishing email using dynamic evolving neural network based on reinforcement learning," *Decis. Support Syst.*, vol. 107, pp. 88–102, Mar. 2018.
- [42] B. Kuchipudi, R. T. Nannapaneni, and Q. Liao, "Adversarial machine learning for spam filters," in *Proc. 15th Int. Conf. Availability, Rel. Secur.*, New York, NY, USA, Aug. 2020, pp. 1–6.
- [43] C. Wang, D. Zhang, S. Huang, X. Li, and L. Ding, "Crafting adversarial email content against machine learning based spam email detection," in *Proc. Int. Symp. Adv. Secur. Softw. Syst.*, New York, NY, USA, May 2021, pp. 23–28.
- [44] F. Concone, S. Gaglio, A. Giammanco, G. L. Re, and M. Morana, "AdverSPAM: Adversarial SPAM account manipulation in online social networks," *ACM Trans. Privacy Secur.*, vol. 27, no. 2, pp. 1–31, May 2024.
- [45] E. Begoli, M. Mahbub, L. Passarella, and S. Srinivasan, "A compound data poisoning technique with significant adversarial effects on transformer-based sentiment classification tasks," *J. Data Inf. Qual.*, vol. 16, no. 4, pp. 1–15, Dec. 2024.
- [46] Q. Cheng, A. Xu, X. Li, and L. Ding, "Adversarial email generation against spam detection models through feature perturbation," in *Proc. IEEE Int. Conf. Assured Autonomy (ICAA)*, Fajardo, PR, USA, Mar. 2022, pp. 83–92.
- [47] X. Ma and W. Shi, "AESMOTe: Adversarial reinforcement learning with SMOTe for anomaly detection," *IEEE Trans. Netw. Sci. Eng.*, vol. 8, no. 2, pp. 943–956, Apr. 2021.
- [48] M. Basavaraju and D. R. Prabhakar, "A novel method of spam mail detection using text based clustering approach," *Int. J. Comput. Appl.*, vol. 5, no. 4, pp. 15–25, Aug. 2010.
- [49] C. Laorden, X. Ugarte-Pedrero, I. Santos, B. Sanz, J. Nieves, and P. G. Bringas, "Study on the effectiveness of anomaly detection for spam filtering," *Inf. Sci.*, vol. 277, pp. 421–444, Sep. 2014.

- [50] S. Halder, R. Tiwari, and A. Sprague, "Information extraction from spam emails using stylistic and semantic features to identify spammers," in *Proc. IEEE Int. Conf. Inf. Reuse Integr.*, Aug. 2011, pp. 104–107.
- [51] S. Sedhai and A. Sun, "Semi-supervised spam detection in Twitter stream," *IEEE Trans. Computat. Social Syst.*, vol. 5, no. 1, pp. 169–175, Mar. 2018.
- [52] S. Y. Yerima and A. Bashar, "Semi-supervised novelty detection with one class SVM for SMS spam detection," in *Proc. 29th Int. Conf. Syst., Signals Image Process. (IWSSIP)*, Jun. 2022, pp. 1–4.
- [53] J. Large, J. Lines, and A. Bagnall, "The heterogeneous ensembles of standard classification algorithms (HESCA): The whole is greater than the sum of its parts," 2017, *arXiv:1710.09220*.
- [54] W. Wang, "Heterogeneous Bayesian ensembles for classifying spam emails," in *Proc. Int. Joint Conf. Neural Netw. (IJCNN)*, Jul. 2010, pp. 1–8.
- [55] S. Nizamani, N. Memon, M. Glasdam, and D. D. Nguyen, "Detection of fraudulent emails by employing advanced feature abundance," *Egyptian Informat. J.*, vol. 15, no. 3, pp. 169–174, Nov. 2014.
- [56] R. Islam and J. Abawajy, "A multi-tier phishing detection and filtering approach," *J. Netw. Comput. Appl.*, vol. 36, no. 1, pp. 324–335, Jan. 2013.
- [57] M. Diale, C. Van Der Walt, T. Celik, and A. Modupe, "Feature selection and support vector machine hyper-parameter optimisation for spam detection," in *Proc. Pattern Recognit. Assoc. South Afr. Robot. Mechatronics Int. Conf. (PRASA-RobMech)*, Nov. 2016, pp. 1–7.
- [58] O. Amayri and N. Bouguila, "Content-based spam filtering using hybrid generative discriminative learning of both textual and visual features," in *Proc. IEEE Int. Symp. Circuits Syst. (ISCAS)*, May 2012, pp. 862–865.
- [59] D. Ruano-Ordás, F. Fdez-Riverola, and J. R. Méndez, "Using evolutionary computation for discovering spam patterns from e-mail samples," *Inf. Process. Manage.*, vol. 54, no. 2, pp. 303–317, Mar. 2018.
- [60] Y. Zhu and Y. Tan, "Extracting discriminative information from e-mail for spam detection inspired by immune system," in *Proc. IEEE Congr. Evol. Comput.*, Jul. 2010, pp. 1–7.
- [61] A. M. Al-Zoubi, A. M. Mora, and H. Faris, "A multilingual spam reviews detection based on pre-trained word embedding and weighted swarm support vector machines," *IEEE Access*, vol. 11, pp. 72250–72271, 2023.
- [62] S. Wang, X. Zhang, Y. Cheng, F. Jiang, W. Yu, and J. Peng, "A fast content-based spam filtering algorithm with fuzzy-SVM and K-means," in *Proc. IEEE Int. Conf. Big Data Smart Comput. (BigComp)*, Shanghai, China, Jan. 2018, pp. 301–307.
- [63] R. K. Kaliyar, A. Goswami, and P. Narang, "FakeBERT: Fake news detection in social media with a BERT-based deep learning approach," *Multimedia Tools Appl.*, vol. 80, no. 8, pp. 11765–11788, Mar. 2021.
- [64] *Enron Email Dataset*. Accessed: 2024. [Online]. Available: <http://www2.cs.cmu.edu/~enron/>
- [65] *SpamAssassin Public Corpus*. Accessed: 2024. [Online]. Available: <https://www.kaggle.com/datasets/beatoa/spamassassin-public-corpus/data>
- [66] *SpamBase*. Accessed: 2024. [Online]. Available: <https://www.kaggle.com/datasets/colormap/spambase>
- [67] *CSDMC2010 SPAM Corpus*. Accessed: 2024. [Online]. Available: <https://github.com/zrz1996/Spam-Email-Classifer-DataSet>
- [68] R. Liaw, E. Liang, R. Nishihara, P. Moritz, J. E. Gonzalez, and I. Stoica, "Tune: A research platform for distributed model selection and training," 2018, *arXiv:1807.05118*.
- [69] I. Loshchilov and F. Hutter, "Decoupled weight decay regularization," 2017, *arXiv:1711.05101*.
- [70] I. Tsimperidis, P. Yoo, K. Taha, A. Mylonas, and V. Katos, "R2BN: An adaptive model for keystroke-dynamics-based educational level classification," *IEEE Trans. Cybern.*, vol. 50, no. 2, pp. 525–535, Feb. 2020.



KAMAL TAHA (Senior Member, IEEE) received the Ph.D. degree in computer science from The University of Texas at Arlington, USA. He was an Engineering Specialist with Seagate Technology, USA, from 1996 to 2005 (Seagate is a leading computer disc drive manufacturer in USA). He was as an Instructor of computer science at The University of Texas at Arlington, from August 2008 to August 2010. He has been an Associate Professor with the Department of Electrical and Computer Engineering, Khalifa University, United Arab Emirates, since 2010. He has over 120 refereed publications that have appeared in prestigious top ranked journals, conference proceedings, and book chapters. Over 40 of his publications have appeared in IEEE TRANSACTIONS. His research interests include information retrieval, data mining, databases, bioinformatics, information forensics and security, and defect characterization of semiconductor wafers, with an emphasis on making data retrieval and exploration in emerging applications more effective, efficient, and robust. He is a member of the Program Committee, an editorial board, and a review panel for a number of international conferences and journals, some of which are IEEE and ACM journals.

...